Student Management System — Project Report

- **Project Type:** Java Console Application
- **Technologies:** Java 21, JDBC, SQLite, Maven
- **Prepared By:** Deepu Bhardwaj
- **Course / Department:** B.Tech – CSE (AI & DS)
- **Institution:** GLA University, Mathura
- **Academic Session:** 2024–2028

---

1. Abstract

The Student Management System is a menu-driven Java application that stores and manages student information in a local SQLite database. It supports adding, viewing, searching, updating, and deleting student records. A dashboard displays total students, average marks, and the top performer. JDBC provides database access and Maven manages dependencies and builds.

2. Problem statement

Manual student-record management can lead to repeated data entry, inconsistent information, and difficulty locating or updating records. This project provides a local application that stores records in a structured database and offers common record-management operations through a command-line menu.

3. Objectives

- Implement create, read, update, search, and delete operations.
- Persist records in SQLite.
- Validate required fields, email format, age, and marks.
- Prevent duplicate emails and invalid values through database constraints.
- Calculate dashboard statistics.
- Demonstrate OOP, JDBC, SQL, exception handling, and Maven.

4. Scope

The project is intended for educational and small-scale local use. It is a console application using one local SQLite database file. It does not currently include a GUI, authentication, network access, or multi-user administration.

5. Technology stack

Technology	Purpose
Java 21	Application logic and object-oriented model
JDBC	Database connectivity
SQLite	Local persistent storage

Technology	Purpose
Maven	Dependency management and build/run commands
VS Code / Java IDE	Development environment

6. Functional requirements

1. Add a student with name, email, course, age, and marks.
2. List all students in a readable table.
3. Search by student ID or partial name.
4. Update student details; blank optional inputs retain current values.
5. Delete a student only after the user types `YES`.
6. Display total students, average marks, and top performer.
7. Validate inputs and display error messages.

7. System design and class responsibilities

- **Main:** menu, input validation, and user interaction.
- **Student:** data model for a student record.
- **Database:** SQLite connection and table initialization.
- **StudentRepository:** SQL queries, CRUD, and dashboard statistics.

Separating interaction, model, database setup, and persistence improves readability and maintainability.

8. Database design

Table: `students`

Column	SQLite type	Constraint / purpose
`id`	INTEGER	Primary key, auto-increment
`name`	TEXT	Required, cannot be blank
`email`	TEXT	Required, unique, case-insensitive
`course`	TEXT	Required, cannot be blank
`age`	INTEGER	Between 16 and 100
`marks`	REAL	Between 0 and 100
`created_at`	TEXT	SQLite current timestamp by default

The table is created automatically at startup. The local database file is `student_management.db`.

9. Workflow

1. Start the application.
2. Initialize the SQLite database and create the table if required.
3. Display the main menu.
4. Read the selected operation and requested information.

5. Validate input.
6. Execute parameterized SQL using JDBC.
7. Display the result and return to the menu.
8. Exit; saved records remain in the database.

10. Validation and reliability

- Name and course cannot be blank.
- Email is checked against a basic email-format pattern.
- Age must be between 16 and 100.
- Marks must be between 0 and 100; non-finite values are rejected.
- SQLite enforces email uniqueness and range constraints.
- Prepared statements are used for user-supplied values.
- Try-with-resources closes JDBC resources.
- Deletion requires explicit confirmation.

11. Testing plan

Test case	Expected result
Add valid student	Record saved and ID displayed
View records	Students displayed in a table
Search by ID / partial name	Matching record(s) displayed
Update a record	Updated values saved
Blank optional update field	Existing value retained
Duplicate email	Database rejects duplicate
Invalid age or marks	Input rejected
Delete with `YES`	Record deleted
Delete without `YES`	Deletion cancelled
Dashboard	Count, average, top performer displayed
Restart application	Saved records remain

This is a test plan. Mark cases as passed only after executing them in your environment.

12. Build and run

Prerequisites: JDK 21 or later and Apache Maven. From the project root (the directory containing `pom.xml`), run:

```
java -version
mvn -version
```

```
mvn clean compile
mvn exec:java
```

The first build may need internet access to download dependencies.

13. Limitations

- Command-line interface only.
- Data is stored in a local database file.
- No authentication, authorization, cloud backup, or multi-user interface.
- Email validation checks basic format only; it does not verify that an address exists.
- This is an educational project, not a production-grade system.

14. Future enhancements

- Add a GUI or web interface.
- Add sorting, pagination, and CSV/PDF export.
- Add automated unit and integration tests.
- Add login, roles, and audit history.
- Add backups, import/export, and additional reports.

15. Conclusion

The Student Management System demonstrates a structured approach to student-record management using Java, JDBC, SQLite, and Maven. It implements CRUD operations, validation, database constraints, and summary statistics. It provides a foundation for future improvements such as a GUI, automated tests, and expanded reporting.

References

- Java documentation: https://docs.oracle.com/en/java/
- Maven documentation: https://maven.apache.org/guides/
- SQLite documentation: https://www.sqlite.org/docs.html
- Xerial SQLite JDBC: https://github.com/xerial/sqlite-jdbc